Team Workplan — Vertical Ownership & Execution Phases

This document turns the validated data structure choices (doc 05) and system architecture (doc 02) into an actual three-person execution plan: who builds what, in what order, and how the pieces come together. It supersedes doc 00's "ownership to be finalized after discussion" note — this *is* that discussion, written down, pending final confirmation by all three members at a team sync.

---

1. Status: the data structure question is closed

Every pick below was benchmarked from scratch against every realistic alternative (see `benchmark/results/RESULTS.md`, Runs 1–11, and doc 05's "Benchmark Evidence" subsections). This is no longer a design-stage prediction — it is a measured result. Nothing in Phases 2 onward should re-open this question.

Subsystem	Final data structure	Confirmed by
Order book price list	**Radix Tree** (16-ary)	Run 1, 2, 10, 11
Order queue (FIFO + cancel)	**Doubly Linked List + Hash Map**	Run 6
Currency arbitrage scanner	**Directed Graph + Bellman-Ford**	Run 3, 4, 5
Trader account cache	**Hash Map + LRU**	Run 7
Currency ticker lookup/autocomplete	**Trie**	Run 8
Trade log compression	**Huffman Coding**	Run 9

Two optional, non-blocking items remain open (do not let them delay Phase 2+): Ladder v2b (2-3 tree / B-tree / B+ — syllabus coverage, not a live candidate) and a deeper-book adversarial queue run. Pick these up opportunistically if time allows later in the roadmap.

2. Backend module breakdown

Six independent modules, plus one connective layer, matching doc 02's architecture:

#	Module	Owns	Talks to
1	**Exchange Engine**	Radix Tree price ladder, DLL+Hash order queue, Price-Time matching loop	Gateway (orders in/out), Logbook (completed trades)
2	**Scanner**	Directed graph, Bellman-Ford negative-cycle detection	Live Feed (rates in), Gateway (alerts out)
3	**Live Feed**	Market data ingestion (doc 06: Finnhub WebSocket primary, Frankfurter fallback)	Scanner (feeds it rates)
4	**Vault**	Hash Map + LRU account cache, persistence (database — see the earlier database discussion)	Exchange Engine (balance checks), Gateway
5	**Directory**	Trie — ticker validation and autocomplete	Gateway, Exchange Engine (validates pairs on order submit)
6	**Logbook**	Huffman compression of completed trade records	Exchange Engine (receives trades), archival storage
—	**Gateway**	REST endpoints + WebSocket broadcast (full contract already specified in doc 02 §4)	Everything — this is what the frontend actually calls

3. Suggested three-way split (confirm at team sync — not final until all three agree)

Each person owns one **complete vertical slice**: their backend module(s) *and* the webpage that shows it off. This keeps individual contribution clean for grading and lets each person demo something finished on their own.

Slice A — The Exchange

- **Backend:** Module 1 (Exchange Engine) + the order-related Gateway endpoints (`POST /api/orders`, `DELETE /api/orders/:id`, `GET /api/orderbook/:pair`, `BOOK_UPDATE` broadcast)
- **Frontend:** Live order book depth visualizer (bid/ask ladder + cumulative depth chart), order placement playground
- **Also owns:** the 5,000-order stress-test simulator (it's testing this person's own engine)

Slice B — The Scanner

- **Backend:** Module 2 (Scanner) + Module 3 (Live Feed) + the arbitrage Gateway endpoints (`GET /api/arbitrage/scan`, `LOOPHOLE_ALERT` broadcast)
- **Frontend:** Interactive force-directed currency graph with glowing arbitrage cycles

Slice C — The Vault & Directory

- **Backend:** Module 4 (Vault) + Module 5 (Directory) + Module 6 (Logbook)
- **Frontend:** Account/margin panel, ticker search & autocomplete widget

Proposed owner names: left blank intentionally — assign at your next sync based on who's already most comfortable with which part (e.g. whoever built/verified a given benchmark module during the DeepSeek handoff has a head start on that slice's logic).

---

4. Phases from here

Phase 1 (data structure validation) is **done**, ahead of doc 04's original Week 1–3 schedule, since the benchmark work already proved every pick with real code and real measurements. The phases below cover everything after that, and can be compressed into doc 04's remaining Week 4–14 window.

Phase 2 — Graduate the code (short)

Each person takes their own slice's data structure(s) out of `benchmark/src/` and moves them into the real application source tree, cleaning up anything that was benchmark-only scaffolding (timing hooks, oracle comparison code). The *logic* is already proven — this is porting, not redesigning.

Phase 3 — Build each backend module + its API

Each person wires their data structure(s) to real HTTP/WebSocket endpoints per doc 02 §4's

contract. **Freeze the API shapes before Phase 5 starts** — the frontend work in Phase 5 depends on these not moving.

Phase 4 — Connect the real external pieces

- Slice B: wire up the actual Finnhub WebSocket connection (doc 06) — this is the one module with an external dependency and failure modes (reconnect/fallback logic), budget extra time here.
- Slice C: wire up the actual database for account persistence.

Phase 5 — Build the frontend

Each person builds their own slice's webpage, against the now-frozen API from Phase 3. These can happen in parallel across all three people once Phase 3 is frozen.

Phase 6 — Integration

Put all three slices together into one running application. **This is where cross-slice bugs appear — budget real calendar time for this, historically the most underestimated phase in any multi-person project.** Suggested integration checkpoints: - Does an order placed in Slice A correctly validate its ticker against Slice C's Directory? - Does a completed trade in Slice A correctly reach Slice C's Logbook? - Does Slice B's scanner run independently without blocking Slice A's matching loop?

Phase 7 — Polish, report, demo rehearsal

Final benchmarking pass if time allows (the two open items from §1), writing the project report, and a full dry-run of the live demo — including a rehearsed answer for "why this data structure" per subsystem, which doc 05 already gives you verbatim.

---

5. One rule worth stating explicitly

No slice re-opens the data structure decision in §1 without new benchmark evidence. If Phase 3 or 6 surfaces a real problem with a pick (not just "I'd rather use X"), the fix is to extend the benchmark harness and measure it — the same process that produced every decision in this document — not to swap structures on a hunch.